Using Local Storage With Widgets

Brief Description

HTML Web Storage API which includes `localStorage` and `sessionStorage`, is a standard feature in modern web browsers that allows web applications to store data persistently on the user's computer. Local Storage is essentially a key-value store, offering a mechanism to store significant amounts of data (typically 5–10 MB) without any expiry date. This means the data remains available even after the user closes and reopens the browser or restarts the computer.

Usage Motivation

In the context of iCUE HTML widgets, a developer can utilize Local Storage to load and save data between iCUE sessions. This data is required by the widget HTML page for correct display and rendering but is not essential for the core functionality of the iCUE itself.

Data Structure and Keying

Each widget possesses a unique identifier (QUuid) which serves as the key for a JSON object stored within Local Storage. This JSON object holds all the persisted properties and settings specific to that widget. The value of this unique ID is obtained from the variable `uniqueId`.

Example

The code snippet below illustrates how to retrieve a specific value from the widget's properties within the stored JSON object:

```
const widgetId = uniqueId;
```

```
const storedData = localStorage.getItem(widgetId);

if (storedData) {
    const properties = JSON.parse(storedData);
    const savedValue = properties.propertyName;

    console.log(`Loaded value for propertyName: ${savedValue}`);
}
```

To save a specific value, simply update the relevant JSON object, stringify this object, and then store the resulting string in Local Storage using the widget's `uniqueId` as the key:

```
const widgetId = uniqueId;

const storedData = localStorage.getItem(widgetId);
let properties = storedData ? JSON.parse(storedData) : {};

properties.propertyName = newValue;

const updatedDataString = JSON.stringify(properties);

localStorage.setItem(widgetId, updatedDataString);

console.log(`Successfully saved new value for propertyName: ${newValue}`);
```

A complete example of a widget for Xeneon Edge that implements the Local Storage save and load mechanism can be seen below:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>'Counter Storage'</title>
        <style>
        * {
            box-sizing: border-box;
        }

        body {
            margin: 0;
            height: 100vh;
            display: flex;
            justify-content: center;
```

```
            align-items: center;
            font-family: Arial, sans-serif;
        }

        .container {
            width: 100%;
            display: flex;
            align-items: center;
            justify-content: space-evenly;
            padding: 16px;
        }

        .circle-button {
            width: 33vw;
            max-width: 220px;
            aspect-ratio: 1 / 1;
            border-radius: 50%;
            border: none;
            background: #007bff;
            color: white;
            font-size: clamp(40px, 8vw, 72px);
            cursor: pointer;
            transition:
                transform 0.15s ease,
                background 0.15s ease;
            display: flex;
            align-items: center;
            justify-content: center;
            user-select: none;
        }

        .circle-button:hover {
            background: #0064d6;
        }

        .circle-button:active {
            transform: scale(0.98);
        }

        .counter {
            min-width: 90px;
            text-align: center;
            font-size: clamp(36px, 7vw, 64px);
            font-weight: bold;
            color: white;
        }
        </style>
    </head>
    <body>
```

```
<div class="container">
    <button id="decrementBtn" class="circle-button">-</button>

    <div id="counter" class="counter">0</div>

    <button id="incrementBtn" class="circle-button">+</button>
</div>

<script>
const MAX_VALUE = 99;
const MIN_VALUE = 0;
const COUNTER_KEY = "counter-widget";
const WIDGET_KEY = uniqueId;

let count = 0;

const counterElement = document.getElementById("counter");
const incrementBtn = document.getElementById("incrementBtn");
const decrementBtn = document.getElementById("decrementBtn");

function saveCount(newValue) {
    count = newValue;

    jsonStorage[COUNTER_KEY] = count;
    const updatedDataString = JSON.stringify(jsonStorage);

    localStorage.setItem(WIDGET_KEY, updatedDataString);
}

function render() {
    counterElement.textContent = count;
}

function loadValue() {
    const widgetStorage = localStorage.getItem(WIDGET_KEY);
    if (widgetStorage) {
        try {
            jsonStorage = JSON.parse(widgetStorage);
        } catch (e) {
            jsonStorage = {};
        }
    } else {
        jsonStorage = {};
    }
    const storedValue = jsonStorage[COUNTER_KEY];

    if (storedValue !== undefined) {
        const currentValue = parseInt(storedValue, 10) || 0;
        count = Math.min(Math.max(currentValue, MIN_VALUE), MAX_VALUE);
```

```
            }
        }

        incrementBtn.addEventListener("click", () => {
            if (count < MAX_VALUE) {
                saveCount(count + 1);
                render();
            }
        });

        decrementBtn.addEventListener("click", () => {
            if (count > MIN_VALUE) {
                saveCount(count - 1);
                render();
            }
        });

        window.addEventListener("storage", (event) => {
            if (event.key === WIDGET_KEY) {
                loadValue();
                render();
            }
        });

        loadValue();
        render();
        </script>
    </body>
</html>
```